

ATHARAV'S INDIVIDUAL TASK SHEET

Role: Full-Stack Developer (Team B - Frontend & Face Recognition Lead)

Team Partner: Umesh

Timeline: 3 Months (90 Days)

Total Hours: 864 hours (72 hours/week × 12 weeks)

Working Days: Monday to Saturday (12 hours/day)

Rest Day: Sunday (Mandatory - NO CODING!)

YOUR PROFILE & RESPONSIBILITIES

Primary Focus Areas:

1. **Frontend Web Development** (40% time)
 - Next.js web application
 - Teacher and Admin portals
 - Complex UIs (test creation, analytics dashboards)
 - Responsive design
2. **Face Recognition System** (30% time)
 - **YOUR SPECIALTY - Most critical technical challenge!**
 - Python FastAPI service for face recognition
 - OpenCV and face_recognition library
 - Face detection, encoding, matching algorithms
 - Integration with attendance system
3. **Frontend Architecture** (15% time)
 - Component library design
 - State management (Zustand)
 - API integration patterns
 - Performance optimization
4. **n8n Workflow Setup** (15% time)
 - Paper generation automation
 - Workflow design and testing
 - Integration with backend APIs

Your Strengths (Leverage These):

- Strong problem-solving skills
- Good understanding of algorithms
- Visual design sense
- Full-stack capabilities

What You'll Learn:

- **Advanced Computer Vision** (Face Recognition)
- **Machine Learning** Integration
- Next.js 14 with App Router
- Professional UI/UX development
- Workflow automation (n8n)
- Production-grade frontend architecture



MONTH 1: FOUNDATION (WEEKS 1-4)

WEEK 1: SETUP & AUTHENTICATION UI (72 hours)

Your Focus: Build login/signup pages, set up frontend

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Project Setup & Design System

Hour-by-Hour Breakdown:

- **Hours 1-6:** Next.js project setup

[] Create Next.js 14 project:

```
npx create-next-app@latest coaching-app-web --typescript --tailwind --app
```

-
- [] Install dependencies:
 - shadcn/ui components
 - Zustand (state management)
 - TanStack Query (data fetching)
 - Recharts (for analytics)
 - zod (validation)
- [] Configure Tailwind CSS

[] Set up folder structure:

```
/app /auth (login, signup pages) /dashboard (protected routes) /admin (admin portal)
/teacher (teacher portal)/components /ui (shadcn components) /custom (our custom
components)/lib (utilities, API client)/hooks (custom React hooks)
```

-
- **Hours 7-12:** Design system setup

[] Install shadcn/ui:
npx shadcn-ui@latest init

-
- [] Install core components:
 - Button, Input, Card, Dialog, Dropdown
 - Table, Tabs, Toast, Avatar
- [] Customize theme colors (primary, secondary, accent)
- [] Create design tokens file (spacing, colors, fonts)
- [] Test components
- **Hours 13-18:** Supabase client setup

- [] Install @supabase/supabase-js

[] Create Supabase client:

```
// lib/supabase.ts
import { createClient } from '@supabase/supabase-js';
export const supabase = createClient( process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!);
```

-
- [] Set up environment variables (.env.local)
- [] Test connection to Supabase
- **Hours 19-24:** Layout components
 - [] Create main layout component
 - [] Navigation bar (with logo, menu items)
 - [] Sidebar component (for dashboards)
 - [] Footer component
 - [] Responsive design (mobile, tablet, desktop)

Deliverables Day 1-2:

- ☒ Next.js project set up with Tailwind + shadcn/ui
- ☒ Design system configured
- ☒ Layout components ready

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Authentication Pages

- **Hours 1-8:** Login page
 - [] Create /app/auth/login/page.tsx

- ☐ Beautiful login form:
 - Email input
 - Password input (with show/hide)
 - Remember me checkbox
 - Forgot password link
 - Login button
- ☐ Form validation (zod schema)
- ☐ Error messages
- ☐ Loading state
- **Hours 9-16:** Signup/Register page
 - ☐ Create /app/auth/signup/page.tsx
 - ☐ Multi-step form:
 - Step 1: Choose role (Student/Teacher/Parent)
 - Step 2: Basic info (name, email, phone)
 - Step 3: Role-specific info (class, subject, etc.)
 - ☐ Progress indicator
 - ☐ Form validation per step
 - ☐ Success message
- **Hours 17-24:** Auth integration
 - ☐ Integrate with Supabase Auth APIs

☐ Handle login:

```
const handleLogin = async (email: string, password: string) => {
  const { data, error } = await supabase.auth.signInWithPassword({
    email,
    password
  });
  if (error) { // Show error toast }
  else { // Redirect to dashboard router.push('/dashboard'); }
};
```

-
- ☐ Handle signup
- ☐ Store auth state in Zustand
- ☐ Protected routes (middleware)

Deliverables Day 3-4:

- ☒ Login and signup pages working
- ☒ Authentication flow integrated

Day 5-6 (Friday-Saturday) - 24 hours

Task: Dashboard Layouts

- **Hours 1-12:** Admin dashboard
 - ☐ Create /app/admin/page.tsx
 - ☐ Dashboard layout:
 - Sidebar with menu (Students, Teachers, Tests, Reports)

- Top bar with user profile, notifications
 - Main content area
- [] Dashboard cards (metrics):
 - Total students
 - Total teachers
 - Tests this month
 - Revenue
- [] Quick actions section
- **Hours 13-20: Teacher dashboard**
 - [] Create /app/teacher/page.tsx
 - [] Similar layout to admin
 - [] Teacher-specific cards:
 - My classes
 - Upcoming tests
 - Pending evaluations
 - Attendance to mark
 - [] Quick links (create test, upload questions)
- **Hours 21-24: Student dashboard**
 - [] Create /app/student/page.tsx
 - [] Clean, student-friendly UI
 - [] Dashboard cards:
 - Upcoming tests
 - Recent scores
 - Attendance percentage
 - Practice questions
 - [] Week 1 wrap-up and demo

Week 1 Complete! 🎉

- Integration Point: Frontend connected to auth APIs ✅

WEEK 2: QUESTION MANAGEMENT UI (72 hours)

Your Focus: Build complete question management interface

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Question List & Upload

- **Hours 1-15: Question list page**
 - [] Create /app/teacher/questions/page.tsx
 - [] Questions table with columns:
 - Question text (truncated to 100 chars)
 - Subject, Chapter, Topic

- Difficulty
- Type (MCQ, Numerical)
- Actions (Edit, Delete, Preview)

[] Advanced filters (shadcn/ui Popover):

// Filters: subject, chapter, difficulty, type// Apply button → refresh table

-
- [] Pagination (20 questions per page)
- [] Search bar (search question text)
- [] Sorting (by date, difficulty, marks)
- **Hours 16-27: Question upload page**
 - [] Create /app/teacher/questions/upload/page.tsx
 - [] Drag-and-drop file upload:
 - Accept CSV, Excel files
 - Show file preview (first 5 rows)
 - Upload button
 - [] Template download button (sample CSV)
 - [] Progress bar during upload
 - [] Success/error summary after upload
- **Hours 28-36: Single question creation**
 - [] Create /app/teacher/questions/new/page.tsx
 - [] Form with fields:
 - Question text (rich text editor or textarea)
 - Subject (dropdown)
 - Chapter (dropdown, filtered by subject)
 - Topic (dropdown, filtered by chapter)
 - Difficulty (Easy/Medium/Hard)
 - Question type (MCQ/Numerical/Subjective)
 - Options (A, B, C, D for MCQs)
 - Correct answer
 - Marks
 - Negative marks
 - Explanation
 - Image upload (optional)
 - [] Form validation
 - [] Save button

Deliverables Day 1-3:

-  Complete question management UI
-  Integrated with backend APIs

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Question Preview & Edit

- **Hours 1-18:** Question preview modal
 - ☐ Build beautiful preview component
 - ☐ Show question in exam-like format
 - ☐ Display all metadata (chapter, difficulty, etc.)
 - ☐ Show correct answer (highlighted)
 - ☐ Show explanation
 - ☐ Close button
- **Hours 19-30:** Question edit page
 - ☐ Create `/app/teacher/questions/[id]/edit/page.tsx`
 - ☐ Pre-fill form with existing question data
 - ☐ Allow editing all fields
 - ☐ Update button
 - ☐ Track changes (show "Modified" badge)
- **Hours 31-36:** Delete & bulk operations
 - ☐ Delete confirmation dialog
 - ☐ Bulk select (checkboxes)
 - ☐ Bulk delete
 - ☐ Bulk update (change difficulty, subject)
 - ☐ Week 2 wrap-up

Week 2 Complete! 🎉

- Integration Point: Question management fully functional 
-

WEEK 3: TEST CREATION UI (72 hours)

Your Focus: Build test creation wizard (complex UI)

Day 1-4 (Monday-Thursday) - 48 hours

Task: Test Creation Wizard

- **Hours 1-15:** Step 1 - Basic Info
 - ☐ Create `/app/teacher/tests/create/page.tsx`
 - ☐ Multi-step wizard (use shadcn/ui Tabs or custom)
 - ☐ Step 1: Basic Info
 - Test name
 - Description
 - Test type (Practice/Mock/Chapter/Full)
 - Duration (minutes)
 - Start date & time

- End date & time
 - Instructions (rich text)
 - ☐ Next button (validate before proceeding)
- **Hours 16-30:** Step 2 - Add Questions
 - ☐ Question selector interface
 - ☐ Two modes:
 - **Manual Selection:** Browse and select questions
 - **Auto Selection:** Set criteria (subject, chapter, difficulty, count)
 - ☐ Selected questions list (drag to reorder)
 - ☐ Remove question button
 - ☐ Question preview on hover
 - ☐ Total marks calculation (auto-sum)
- **Hours 31-45:** Step 3 - Configuration
 - ☐ Marks per question (or use default from question)
 - ☐ Negative marking (enable/disable, value)
 - ☐ Shuffling options:
 - Shuffle questions
 - Shuffle options (for MCQs)
 - ☐ Proctoring settings:
 - Enable proctoring
 - Tab switch detection
 - Webcam monitoring
 - Max violations allowed
 - ☐ Next button
- **Hours 46-48:** Step 4 - Student Assignment
 - ☐ Select students to assign:
 - ☐ Entire class/batch
 - ☐ Individual students
 - ☐ Multiple classes
 - ☐ Selected students list
 - ☐ Search students
 - ☐ Finish button

Deliverables Day 1-4:

- ☒ Complete test creation wizard
- ☒ Integrated with backend APIs

Day 5-6 (Friday-Saturday) - 24 hours

Task: Test Management

- **Hours 1-12:** Test list page

- [] Create /app/teacher/tests/page.tsx
- [] Tests table:
 - Test name
 - Type
 - Start date
 - Duration
 - Status (Scheduled/Live/Completed)
 - Assigned students count
 - Actions (Edit, View Results, Delete)
- [] Filters (by status, type, date range)
- [] Status badges (color-coded)
- **Hours 13-20:** Test preview page
 - [] Create /app/teacher/tests/[id]/preview/page.tsx
 - [] Show complete test (as students will see it)
 - [] Question palette
 - [] Timer (simulated)
 - [] All questions with options
 - [] Publish button (if not published)
- **Hours 21-24:** Week 3 wrap-up
 - [] Test entire test creation flow
 - [] Bug fixes
 - [] Demo to team

Week 3 Complete! 🎉

- Integration Point: Teachers can create tests ✅

WEEK 4: STUDENT TEST INTERFACE (72 hours)

Your Focus: Build NTA-pattern exam interface (complex!)

Day 1-4 (Monday-Thursday) - 48 hours

Task: Student Exam Interface

- **Hours 1-12:** Test start page
 - [] Create /app/student/tests/[id]/start/page.tsx
 - [] Test instructions display
 - [] "I am ready" button
 - [] Full-screen enforcement (request full-screen)
 - [] Pre-exam checks:
 - Camera permission (if proctoring)
 - Stable internet check
 - Close other tabs warning

- **Hours 13-30:** Main exam interface

- [] Create /app/student/tests/[id]/attempt/page.tsx

[] **NTA Pattern Layout:**

Name] Test Name		Timer: 02:45:30	[Student
Question 15 of 90		Section: Physics	
Question text here...			◦ Option A
◦ Option B		◦ Option C	◦ Option D
		[Mark for Review] [Clear Response]	
[← Previous]	[Save & Next →]		
Question Palette		[1][2][✓ 3][✓ 4][5][6][△ 7]...	
Legend:		[] Not Visited [✓] Answered	[△]
Marked for Review [] Not Answered			

◦

- **Hours 31-40:** Question navigation

- [] Question palette (grid of buttons):
 - Color codes:
 - Green: Answered
 - Red: Not answered (visited)
 - Purple: Marked for review
 - Gray: Not visited
 - [] Click on palette button → jump to that question
 - [] Previous/Next buttons
 - [] Question counter (15 of 90)

- **Hours 41-48:** Answer handling

- [] Radio buttons for MCQs (single correct)
- [] Checkboxes for multi-correct MCQs
- [] Number input for numerical questions
- [] Auto-save answer (every 10 seconds)
- [] Clear response button
- [] Mark for review button

Deliverables Day 1-4:

- ☒ Complete NTA-pattern exam interface
- ☒ Question navigation working

Day 5-6 (Friday-Saturday) - 24 hours

Task: Timer & Submission

- **Hours 1-10:** Timer implementation
 - ☐ Countdown timer (HH:MM:SS format)
 - ☐ Update every second
 - ☐ Store in localStorage (persist on page reload)
 - ☐ Warning when 5 minutes left (toast notification)
 - ☐ Auto-submit when time = 0
- **Hours 11-18:** Test submission
 - ☐ Submit button (with confirmation dialog)
 - ☐ Show summary before submit:
 - Answered: X
 - Not answered: Y
 - Marked for review: Z
 - ☐ Final submit button
 - ☐ Submission loading state
 - ☐ Redirect to results page
- **Hours 19-24:** Results page
 - ☐ Create /app/student/tests/[id]/results/page.tsx
 - ☐ Show score card:
 - Total score
 - Percentage
 - Rank
 - Time taken
 - ☐ Subject-wise breakdown
 - ☐ View answer sheet button
 - ☐ Week 4 & Month 1 wrap-up

Week 4 Complete! Month 1 Done! 🎉🎉

- Integration Point: Students can take full tests ☒

 17

MONTH 2: AI FEATURES & FACE RECOGNITION (WEEKS 5-8)

WEEK 5: PRACTICE MODE & AI INTEGRATION (72 hours)

Your Focus: Build adaptive learning UI

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Practice Session UI

- **Hours 1-15:** Practice mode interface
 - ☐ Create /app/student/practice/page.tsx
 - ☐ Similar to test interface but simplified
 - ☐ Single question at a time
 - ☐ Submit answer button
 - ☐ Instant feedback:
 - ☒ Correct (show explanation)
 - ☒ Wrong (show correct answer + explanation)
 - ☐ If wrong, show "Practice similar questions" button
- **Hours 16-27:** Similar questions display

☐ When student gets wrong answer:

// Call AI API: GET /api/ai/generate-similar?questionId=xxx// Display 5 similar questions

- ☐
- ☐ Show 5 similar questions (one by one)
- ☐ Track progress (1/5, 2/5, etc.)
- ☐ Celebrate when all 5 correct! 🎉
- **Hours 28-36:** Progress tracking UI
 - ☐ Create /app/student/progress/page.tsx
 - ☐ Show weak topics (red badges)
 - ☐ Show strong topics (green badges)
 - ☐ Radar chart (topic-wise proficiency)
 - ☐ Recommend topics to practice

Deliverables Day 1-3:

- ☒ Adaptive learning UI working
- ☒ Integrated with AI APIs

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Analytics Dashboard for Teachers

- **Hours 1-18:** Teacher analytics page
 - ☐ Create /app/teacher/analytics/page.tsx
 - ☐ Class performance chart (line chart - performance over time)
 - ☐ Subject-wise performance (bar chart)
 - ☐ Top performers list (leaderboard)
 - ☐ Students needing attention (low scorers)
 - ☐ Filters (by class, test, date range)

- **Hours 19-30:** Student detailed analysis
 - ☐ Create /app/teacher/students/[id]/analysis/page.tsx
 - ☐ Student profile (name, photo, class)
 - ☐ Performance trend (last 10 tests)
 - ☐ Topic-wise heatmap (red = weak, green = strong)
 - ☐ Time management analysis (fast/slow)
 - ☐ Recommendations for improvement
- **Hours 31-36:** Week 5 wrap-up
 - ☐ Test all analytics features
 - ☐ Beautiful charts (Recharts library)
 - ☐ Demo to team

Week 5 Complete! 🎉

WEEK 6: FACE RECOGNITION SYSTEM (72 hours)

🔥 **YOUR CRITICAL WEEK - Face Recognition is your specialty!**

Your Focus: Build complete face recognition system (backend + frontend)

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Face Recognition Backend (Python)

- **Hours 1-8:** Python environment setup
 - ☐ Create separate Python project (FastAPI)

☐ Install dependencies:

```
pip install fastapi uvicorn face_recognition opencv-python pillow
```

○

☐ Project structure:

```
/face-recognition-service /api (FastAPI routes) /services (face recognition logic) /models
(ML models if needed) /utils (helper functions)
```

○

- **Hours 9-16:** Face detection

☐ Implement face detection:

```
import face_recognition
import cv2

def detect_faces(image_path):
    image = face_recognition.load_image_file(image_path)
    face_locations = face_recognition.face_locations(image)
    return face_locations # [(top, right, bottom, left)]
```

-
- [] Test with sample images
- [] Handle multiple faces in one image
- [] Handle no face detected (error case)
- **Hours 17-24:** Face encoding

```
[ ] Generate face encoding (128-dimensional vector):
def generate_face_encoding(image_path): image =
face_recognition.load_image_file(image_path) face_encodings =
face_recognition.face_encodings(image) if len(face_encodings) > 0: return
face_encodings[0] # Return first face return None
```

-
- [] Test with multiple images of same person
- [] Ensure consistency (same person → similar encodings)

Deliverables Day 1-2:

-  Python face recognition service set up
-  Face detection and encoding working

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Face Matching & Recognition

- **Hours 1-12:** Face matching algorithm

```
[ ] Compare two face encodings:
def match_faces(known_encoding, test_encoding, tolerance=0.6): results =
face_recognition.compare_faces([known_encoding], test_encoding,
tolerance=tolerance) # Calculate distance (lower = more similar) distance =
face_recognition.face_distance([known_encoding], test_encoding) return {
"match": results[0], "confidence": 1 - distance[0], # Convert distance to confidence %
"distance": distance[0] }
```

-
- **Hours 13-20:** Batch face recognition

```
[ ] Recognize multiple faces in a single image:
def recognize_students(image_path, known_students): # known_students = [ # {"id":
"uuid", "name": "John", "encoding": [0.1, 0.2, ...]}, # ... # ] # Detect all faces in
image # For each face, compare with all known students # Return matches with
confidence scores return recognized_students
```

-

- ☐ Handle partial matches (confidence < 90%)
- ☐ Handle unknown faces
- **Hours 21-24:** FastAPI endpoints
 - ☐ POST /api/face/detect (detect faces in image)
 - ☐ POST /api/face/encode (generate encoding)
 - ☐ POST /api/face/match (match two encodings)
 - ☐ POST /api/face/recognize-batch (recognize multiple students)
 - ☐ Test all endpoints

Deliverables Day 3-4:

- ☒ Face matching algorithm working
 - ☒ FastAPI endpoints ready
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Face Registration UI & Live Recognition

- **Hours 1-12:** Face registration UI
 - ☐ Create /app/student/register-face/page.tsx
 - ☐ Camera access (using getUserMedia API)
 - ☐ Show live camera feed
 - ☐ Capture photo button
 - ☐ Capture 5-10 photos from different angles:
 - Front face
 - Slight left
 - Slight right
 - With smile
 - Neutral expression
 - ☐ Show captured photos (grid)
 - ☐ Delete photo button (if bad quality)
 - ☐ Submit button (send to backend)
 - **Hours 13-20:** Live face recognition UI
 - ☐ Create /app/teacher/attendance/mark/page.tsx
 - ☐ Camera feed (live)
 - ☐ Face detection overlay (draw rectangle around faces)
- ☐ Auto-recognize faces every 2 seconds:
 // Capture frame from video// Send to Python service// Get recognized students// Display names on UI
- ☐ Show confidence score (Green if > 90%, Yellow if 70-90%, Red if < 70%)
 - ☐ Mark attendance button (for recognized students)

- **Hours 21-24:** Week 6 wrap-up
 - ☐ Test face recognition with real faces (you and Umesh!)
 - ☐ Test in different lighting conditions
 - ☐ Optimize for speed (target: < 1 second per recognition)
 - ☐ Demo to team (this is your moment! ★)

Week 6 Complete! Face Recognition Working! 🎉📷

- Integration Point: Automated attendance through face recognition ✅
-

WEEK 7: PROCTORING UI (72 hours)

Your Focus: Build proctoring interface for students and teachers

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Student Proctoring Experience

- **Hours 1-15:** Full-screen enforcement

☐ Modify test interface to enforce full-screen:

```
useEffect(() => { document.documentElement.requestFullscreen();
document.addEventListener('fullscreenchange', handleFullscreenChange);
document.addEventListener('blur', handleWindowBlur); return () => { // cleanup }}, []);
```

- ☐ Detect when full-screen is exited
- ☐ Show warning dialog
- ☐ Log violation
- **Hours 16-27:** Tab switch detection

☐ Use Page Visibility API:

```
document.addEventListener('visibilitychange', () => { if (document.hidden) { // Tab
switched or window minimized logViolation('TAB_SWITCH'); showWarning(); }});
```

- ☐ Show warning toast (Warning 1 of 3)
- ☐ Track violation count
- ☐ Auto-submit after 3 violations
- **Hours 28-36:** Webcam monitoring UI
 - ☐ Small webcam preview (bottom-right corner)
 - ☐ Capture snapshot every 2 minutes
 - ☐ Send to backend for storage

- ☐ Face not detected alert (if student not in frame)
- ☐ Multiple faces detected alert

Deliverables Day 1-3:

- ☒ Proctoring features implemented on test interface
-

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Teacher Live Proctoring Dashboard

- **Hours 1-18:** Live proctoring dashboard
 - ☐ Create /app/teacher/proctor/[testId]/page.tsx
 - ☐ Real-time student grid:
 - Student name
 - Current question number
 - Time remaining
 - Violation count
 - Trust score (100 - decreases with violations)
 - Live webcam feed (thumbnail)
 - ☐ WebSocket connection for real-time updates
 - ☐ Color-coded status (green = good, yellow = warning, red = flagged)
- **Hours 19-30:** Violation alerts

☐ Real-time violation notifications:

"Student John Doe: Tab switch detected (Warning 2/3)"

- ☐ Click on student → show detailed view:
 - Violation timeline
 - Webcam snapshots
 - Current screen view
- ☐ Flag student button (for manual review)
- **Hours 31-36:** Post-exam proctoring report
 - ☐ Create /app/teacher/tests/[id]/proctoring-report/page.tsx
 - ☐ For each student:
 - Total violations
 - Trust score
 - Violation breakdown (tab switches, no face, etc.)
 - Webcam snapshot gallery
 - Recommendation (Pass/Flag for review/Disqualify)
 - ☐ Export report as PDF

Week 7 Complete! Proctoring System Ready! 🎉🔒

WEEK 8: N8N WORKFLOWS & PAPER GENERATION UI (72 hours)

Your Focus: Set up n8n workflows and build paper generation UI

Day 1-3 (Monday-Wednesday) - 36 hours

Task: n8n Setup & Basic Workflows

- **Hours 1-10:** n8n installation

☐ Install n8n:

Option 1: Docker (recommended)
docker run -d --name n8n -p 5678:5678 -v ~/.n8n:/home/node/.n8n n8nio/n8n
Option 2: npm
npm install n8n -g
n8n start

- - ☐ Access n8n UI (<http://localhost:5678>)
 - ☐ Create account and set up
 - ☐ Understand workflow basics (watch tutorial)
- **Hours 11-24:** Simple notification workflow
 - ☐ Create first workflow (practice):
 1. Webhook trigger (test started)
 2. HTTP Request node (get student details)
 3. Email node (send notification)
 - ☐ Test workflow end-to-end
 - ☐ Learn about error handling
- **Hours 25-36:** OCR workflow (basic)
 - ☐ Workflow design:
 1. Webhook trigger (file uploaded)
 2. Google Vision API node (OCR)
 3. OpenAI node (clean text)
 4. HTTP Request node (save to database)
 5. Email node (notify teacher)
 - ☐ Test with sample image
 - ☐ Handle errors (if OCR fails)

Deliverables Day 1-3:

- ☒ n8n installed and configured
- ☒ Basic workflows working

Day 4-6 (Thursday-Saturday) - 36 hours

Task: Paper Generation UI

- **Hours 1-15:** Photo upload UI
 - [] Create /app/teacher/papers/generate-from-photo/page.tsx
 - [] Drag-and-drop photo upload
 - [] Accept: JPG, PNG, PDF
 - [] Show preview of uploaded image
 - [] "Generate Paper" button
 - [] Trigger n8n workflow (webhook)
- **Hours 16-27:** Processing status UI
 - [] Show processing stages:
 - 🕒 Extracting text from image (OCR)...
 - 🕒 Cleaning and formatting...
 - 🕒 Generating Word document...
 - ✅ Done! Download ready
 - [] Use WebSocket or polling to check status
 - [] Progress bar
- **Hours 28-36:** Generated paper preview
 - [] Show preview of generated paper
 - [] Manual edit option (textarea with paper content)
 - [] Regenerate button (if quality is poor)
 - [] Download button (Word/PDF format)
 - [] Week 8 & Month 2 wrap-up

Week 8 Complete! Month 2 Done! 🎉🎉



MONTH 3: POLISH & LAUNCH (WEEKS 9-12)

WEEK 9: ANALYTICS CHARTS & REPORTS UI (72 hours)

Your Focus: Build beautiful analytics dashboards

Day 1-6 (Monday-Saturday) - 72 hours

Task: Comprehensive Analytics UI

- **Hours 1-20:** Performance charts
 - [] Line chart (performance over time)
 - [] Bar chart (subject-wise comparison)
 - [] Pie chart (difficulty-wise accuracy)
 - [] Radar chart (topic-wise proficiency)

- ☐ Use Recharts library
- ☐ Responsive design
- **Hours 21-40: Interactive filters**
 - ☐ Date range picker (last 7 days, 30 days, all time)
 - ☐ Subject filter
 - ☐ Class/batch filter
 - ☐ Apply filters → update all charts
- **Hours 41-60: Export functionality**
 - ☐ Export chart as PNG
 - ☐ Export data as CSV/Excel
 - ☐ Export report as PDF
 - ☐ Print-friendly view
- **Hours 61-72: Mobile responsive**
 - ☐ Make all charts mobile-responsive
 - ☐ Simplified view on small screens
 - ☐ Touch-friendly interactions
 - ☐ Week 9 wrap-up

Week 9 Complete! 🎉

WEEK 10: PAYMENT UI & POLISH (72 hours)

Your Focus: Build payment UI and polish existing features

Day 1-3 (Monday-Wednesday) - 36 hours

Task: Payment & Subscription UI

- **Hours 1-15: Pricing page**
 - ☐ Create /app/pricing/page.tsx
 - ☐ Beautiful pricing cards (Starter, Professional, Enterprise)
 - ☐ Feature comparison table
 - ☐ "Choose Plan" buttons
 - ☐ FAQ section
- **Hours 16-27: Payment checkout**

☐ Integrate Razorpay checkout:

```
const options = { key: process.env.NEXT_PUBLIC_RAZORPAY_KEY, amount: plan.price *
100, // in paise currency: 'INR', name: 'CoachingPro', description: plan.name, handler:
function(response) { // Payment success
verifyPayment(response.razorpay_payment_id); }};const rzp = new
window.Razorpay(options);rzp.open();
```

-
- **Hours 28-36:** Subscription management page
 - ☐ Create /app/admin/subscription/page.tsx
 - ☐ Show current plan
 - ☐ Usage statistics (students added, tests created)
 - ☐ Upgrade/downgrade buttons
 - ☐ Payment history
 - ☐ Download invoices

Deliverables Day 1-3:

- ☒ Payment UI complete
- ☒ Integrated with Razorpay

Day 4-6 (Thursday-Saturday) - 36 hours

Task: UI/UX Polish

- **Hours 1-18:** Fix bugs and improve UX
 - ☐ Go through entire app
 - ☐ Fix visual bugs
 - ☐ Improve loading states (skeleton loaders)
 - ☐ Better error messages
 - ☐ Consistent spacing and colors
- **Hours 19-30:** Performance optimization
 - ☐ Code splitting (lazy load components)
 - ☐ Image optimization (use Next.js Image component)
 - ☐ Reduce bundle size
 - ☐ Lighthouse audit (target: 90+ score)
- **Hours 31-36:** Accessibility
 - ☐ Keyboard navigation
 - ☐ ARIA labels
 - ☐ Color contrast (WCAG AA)
 - ☐ Week 10 wrap-up

Week 10 Complete! 🎉

WEEK 11: MOBILE APP SUPPORT & TESTING (72 hours)

Your Focus: Help Umesh with mobile app and extensive testing

Day 1-6 (Monday-Saturday) - 72 hours

Task: Testing & Quality Assurance

- **Hours 1-20:** Cross-browser testing
 - ☐ Test on Chrome, Firefox, Safari, Edge
 - ☐ Fix browser-specific bugs
 - ☐ Ensure consistent behavior
- **Hours 21-40:** Responsive design testing
 - ☐ Test on desktop (1920x1080, 1366x768)
 - ☐ Test on tablet (iPad, Android tablets)
 - ☐ Test on mobile (iPhone, Android phones)
 - ☐ Fix layout issues
- **Hours 41-60:** User flow testing
 - ☐ Test complete user journeys
 - ☐ Document bugs in spreadsheet
 - ☐ Fix critical bugs
 - ☐ Retest after fixes
- **Hours 61-72:** Help Umesh with mobile app
 - ☐ Review mobile app components
 - ☐ Help with complex UI (test interface on mobile)
 - ☐ Code review

Week 11 Complete! 🎉

WEEK 12: FINAL POLISH & LAUNCH (72 hours)

Your Focus: Final touches and launch preparation

Day 1-2 (Monday-Tuesday) - 24 hours

Task: Production Deployment

- **Hours 1-12:** Deploy to Vercel
 - ☐ Push code to GitHub
 - ☐ Connect Vercel to GitHub repo
 - ☐ Configure environment variables
 - ☐ Deploy to production
 - ☐ Test production URL
- **Hours 13-24:** Post-deployment testing
 - ☐ Test all features in production

- ☐ Check performance (GTmetrix, Lighthouse)
 - ☐ Fix any production-specific issues
-

Day 3-4 (Wednesday-Thursday) - 24 hours

Task: Beta Testing Support

- **Hours 1-12:** Beta tester onboarding
 - ☐ Create demo accounts
 - ☐ Prepare walkthroughs
 - ☐ Conduct onboarding calls
 - **Hours 13-24:** Bug monitoring and fixes
 - ☐ Monitor for bugs
 - ☐ Fix critical issues immediately
 - ☐ Update documentation
-

Day 5-6 (Friday-Saturday) - 24 hours

Task: Launch!

- **Hours 1-12:** Final checks
 - ☐ All features working
 - ☐ No critical bugs
 - ☐ Performance good
 - ☐ SEO optimized
- **Hours 13-24:** Celebrate! 🎉
 - ☐ We launched!
 - ☐ Team celebration
 - ☐ Reflect on learnings

Week 12 Complete! WE LAUNCHED! 🚀🎉

YOUR KEY METRICS

Development:

- ☐ Pages completed: ____ / ____
- ☐ Components built: ____
- ☐ Bugs fixed: ____

Face Recognition:

- ☐ Recognition accuracy: ____%
- ☐ Recognition speed: ____ seconds
- ☐ False positive rate: ____%

Quality:

- ☐ Lighthouse score: ____
 - ☐ Bundle size: ____ KB
-

YOUR TOOLS

Daily:

- VS Code + React DevTools
- Chrome DevTools
- Figma (design reference)
- GitHub

Face Recognition:

- Python + Jupyter Notebook (testing)
 - OpenCV (debugging)
 - Postman (test Python APIs)
-

PERSONAL GOALS

Technical:

- Master Next.js 14 (App Router)
- Become expert in Computer Vision
- Build production-grade UIs
- Performance optimization skills

Career:

- Build impressive portfolio project
 - Face recognition expertise (unique skill!)
 - Full-stack capabilities
 - Real product launch experience
-

FINAL CHECKLIST

By Week 4:

- ☐ Complete frontend foundation
- ☐ Test interface working

By Week 6:

- ☐ **FACE RECOGNITION WORKING!** ★

By Week 12:

- ☐ Beautiful, fast, responsive UI
 - ☐ Production deployment
 - ☐ **YOU LAUNCHED!** 🚀
-

ATHARAV'S MANTRA

"I build interfaces that users love."

Daily Affirmations:

- "Face recognition is my superpower"
- "I create pixel-perfect UIs"
- "I ask Aditya when stuck (no shame)"
- "I test my code on multiple devices"
- "I rest on Sundays"

Remember:

- Face recognition is hard but you got this!
 - Your UI is what users will see and judge
 - Attention to detail matters
 - Performance matters
 - You're building something special! 💪
-

LET'S BUILD AMAZING UI, ATHARAV! 🚀

You're going to build the face recognition system - how cool is that?! 📷